2. The client application sends an API request to the LLM API, which includes both textual prompts and visual information (e.g., images or image URLs).
3. The API request, with the textual prompts and the binary image data, is sent to the LLM API.
4. The LLM API receives the request and forwards it to the appropriate language model, such as GPT-4o or GPT-4 Turbo or Gemini.
5. The language model processes the visual information (binary image data) through its visual understanding capabilities, analyzing the images and extracting relevant features or semantics.
6. The model then combines the visual understanding with the textual prompts to generate a relevant response or description.
7. The generated response is sent back to the client application through the API response.

What I don't show exactly in this workflow is that the client application is responsible for encoding the image file into a binary format (e.g., using the FileReader API or other techniques) before including it in the API request. The binary image data can be sent as part of the request body or as a separate file upload, depending on the API requirements.

However, as we will see in the following section, the implementation is straightforward. Before we dive into those details, let's clarify some caveats when using this feature.

WHAT ARE THE CAVEATS?

Sending multimedia files along with the prompt is a useful feature, but it has limitations. While some APIs may support sending multiple images as part of a single prompt, it is generally recommended to send one image at a time. Sending multiple images can increase complexity and processing time and may not always yield optimal results. If you need to provide multiple images as context, it's better to break them down into separate prompts or requests.

Another problem is file limitations. Some providers, like OpenAI, allow images up to a certain file size limit (20 MB for OpenAI), while others might accept smaller files. It's advisable to provide high-quality images with lossless compression. If you are hosting your own models, this file size limitation might not be a problem, of course.

Additionally, it's important to consider image content. Models may struggle with images that have low contrast, poor lighting, or excessive noise, which can negatively affect their ability to accurately interpret the content. Ensuring that images are clear and well lit can significantly improve the model's performance and the quality of the generated responses.

Now that we've covered the details of integrating multimedia in prompts, the implementation part is minimal, since we already have all the necessary pieces in place with the Vercel AI SDK. The most technical parts are explained in the next section.